

Szegedi Tudományegyetem
Informatikai Tanszékcsoport

SZAKDOLGOZAT

Dinnyés Balázs

2013

**Szegedi Tudományegyetem
Informatikai Tanszékcsoport**

Páros munka ütemezési probléma algoritmusai

Szakdolgozat

Készítette:
Dinnyés Balázs
informatika szakos
hallgató

Témavezető:
Dr. Békési József
főiskolai tanár

Szeged
2013

Tartalom

Tartalom.....	3
Feladatkiírás.....	4
Tartalmi összefoglaló.....	4
1. Heurisztikus megoldási algoritmusok.....	5
1.1 Polinomiális futási idejű algoritmusok.....	5
1.1.1 1. Algoritmus	5
1.1.2 2. Algoritmus	6
1.1.3 Újabb heurisztikus megoldás egyedi bemenetekre	8
1.2 Exponenciális futási idejű algoritmusok. Korlátozás és szétválasztás módszere (branch & bound, B&B)	9
1.2.1 Branch and Bound eljárás ismertetése	9
1.2.1. 1. Algoritmus	10
1.2.2. 2. Algoritmus	12
2. Egy optimális eredményt szolgáltató algoritmus, hasonló, de más B&B fával, heurisztikus módszereket alkalmazva, amikkel a fa mérete csökkenthető.	13
2.1 A szétválasztó módszer	14
2.2 Egy korlátozó módszer, az azonosság.....	14
2.3 Egy másik korlátozó módszer, a symmetry breaking	16
2.4 Korlátozás egzaktabb fajtái	17
3. Az eddigi megoldások előzetes értékelése I.	18
4. Egy iskolászerű példa az Utazó ügynök problémája (TSP).....	21
5. Az eddigi megoldások előzetes értékelése II.	23
6. Az 1. exponenciális futási idejű, heurisztikus algoritmus elemzése kombinatorikusan.....	25
7. A leimplementált algoritmusok összehasonlítása, összefoglalása	27
Irodalom jegyzék:	30
Nyilatkozat.....	31
Köszönetnyilvánítás.....	32

Feladatkiírás

Az ütemezési problémák a kombinatorikus optimalizálás egyik jól ismert területét jelentik. Általában az ütemezés célja, hogy adott munkákat ütemezzünk adott gépeken úgy, hogy az utoljára végrehajtandó munka a lehető leghamarabb befejeződjön. A problémának nagyon sokféle változata létezik. A dolgozatban egy viszonylag egyszerűen definiálható változattal kell foglalkozni, amikor a munkák két részből állnak, a részek között fix hosszúságú szünettel, és csupán egyetlen gépet használunk. A dolgozatnak ki kell térni a feladat elméleti vizsgálatára, valamint el kell készíteni bizonyos heurisztikus vagy egzakt algoritmusok implementációját, és empirikus módon történő elemzését.

Tartalmi összefoglaló

A Páros Munka Ütemezésnek, mint a legtöbb algoritmusnak, létezik online és offline változata. Az előző esetben nem ismerjük előre az összes munka paraméterét, hanem, mint egy futószalagon érkezve, egyenként kell nyomban eldöntenünk a sorsukat. Ennél a változatnál nem sok lehetőségünk van, mivel egyetlen páros munkának (továbbiakban: munka) a paramétereit tudjuk csak előre. A legjobban tehát akkor járunk, ha a munkákat sorban elhelyezzük egymás mellett, ha lehet minél hamarabb beágyazva egy másik munkába.

Többek közt ehhez a módszerhez hasonló, mohó választást használok fel, és implementálok le algoritmusaim közt. Az implementáláshoz Java nyelvet használok és Branch and Bound technikával dolgozom.

Az offline változatnál, amit vizsgálunk, több lehetőségünk van. Itt a lehetőségek és egyben a korlátok a munkák részmunkái közt helyet foglaló fix hosszúságú szünetekben rejlenek. Ilyenkor be lehet ékelni egy munkába akár több munka részmunkáját, akár egy egész munkát vagy többet, vagy ezek kombinációját.

E dolgozat keretein belül ismertetésre kerül három polinom, $O(n^2)$ futási idejű algoritmus, majd két exponenciális $O(2^n)$ idejű algoritmus, és egy optimális megoldást adó algoritmus.

A felsoroltak közül csak egy polinom futási idejű algoritmust maradt ki az implementációból, vele csak szemléltetni akartam a dolgozatban, mennyire függ az egyes heurisztikák sikere a bemenetek tulajdonságaitól.

Az eredmények kimutatták, hogy a kifejlesztett heurisztikus megoldások által szolgáltatott eredmények nagy közelítést mutatnak az optimális megoldáshoz. Csak kevés, speciális esetben érdemes a hosszabb futási idejű optimális megoldást szolgáltató algoritmusokat használni, amire mutatok példát is.

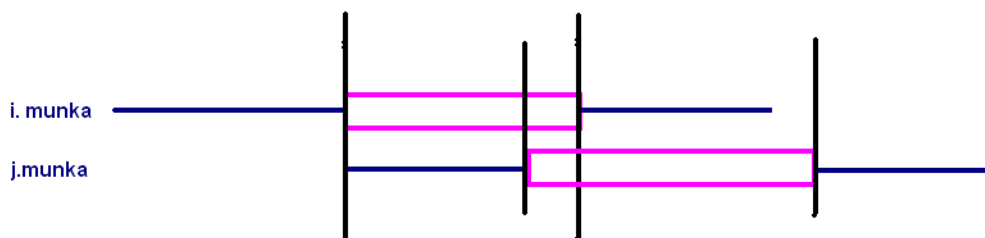
1. Heurisztikus megoldási algoritmusok

1.1 Polinomiális futási idejű algoritmusok

1.1.1 1. Algoritmus

A heurisztikus algoritmus futási ideje $O(n^2)$. Feltételezi, hogy az egyes munkák részmunkaidői és szünetei hasonlóak, mondjuk ez a természetes is, ekkor a legjobb az eredmény.

Veszi egy véletlenszerű sorba helyezésnek a soron következő elemét ($i := (1 \dots n)$) és megnézi, hogy a többi elem ($j := 1 \dots n$) melyike helyezhető el benne a következő ábra szerinti módon:



1.1.1 1. ábra.

A beágyazás menete. Az i . munka a beágyazó j . a beágyazott.

Páros munka ütemezés probléma algoritmusai

Az ábra szerinti esetben a beágyazás sikeres. A vízszintes vonalak a részmunkák a téglalapok a szünetek.

Látható, hogy a beágyazás abban a matematika formában tud megvalósulni, ha a beágyazó szünete nagyobb, mint a beágyazott első részmunkája, és ezzel együtt a beágyazott első részmunkájának és a szünetének az összege nagyobb, mint a beágyazó szünetének és a második részmunka idejének az összege. Vagyis:

$$(szünet_i \geq munka_j_1) \ \&\& \ (szünet_i + munka_i_2 \leq munka_j_1 + szünet_j).$$

Ha ebbe az elembe több más elem is beágyazható - természetesen ezek közül csak egy lesz, hiszen a heurisztika így működik - akkor ezek közül a legkisebb beágyazást veszi. Beágyazás eredménye:

$$\text{Beágyazás} := munka_i_1 + munka_j.$$

Ezután, a következő beágyazó és beágyazott keresése során a fent említett i . és j . munkát már nem veszi figyelembe. Tehát n bemeneti munka során, ha azt vesszük, hogy minden munkát egymással összehasonlítunk, akkor számolhatunk $O(n^2)$ futási idővel. Persze ennek a heurisztikának kevesebb idő is elég, hiszen minden végig ellenőrzés után 2 - vel kevesebbet kell ellenőrizni.

1.1.2 2. Algoritmus

Az általam leimplementált másik polinom futási idejű algoritmus is a fent említett módszerrel ellenőrzi két munka beágyazhatóságát. Viszont ha több lehetőség van, akkor más stratégia alapján választ közülük. Méghozzá így:

$\min((szünet_j - munka_i_2) / (munka_i_1 + munka_j_1 + szünet_j))$. A keletkezett szünet és a beágyazás arányát figyeli az algoritmus.

Lényegében, az algoritmus szerint, hiába kisebb egy beágyazás két beágyazás közül, ha kisebbnek, aránytalanul, nagyobb a keletkezett, valós szünete, például:

Az egyik beágyazás:

Páros munka ütemezés probléma algoritmusai

(az i. munka mindkét beágyazásnál azonos)

$munka_i_1 := 7$

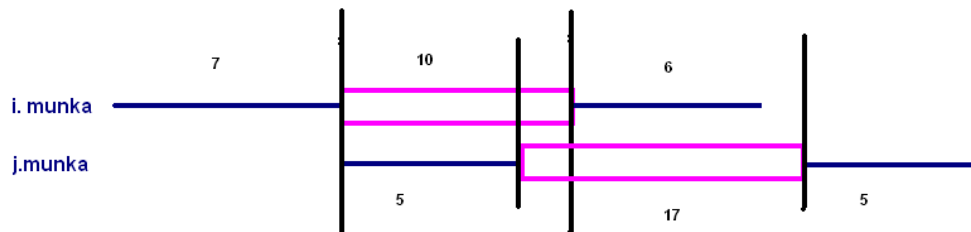
$szünet_i := 10$

$munka_i_2 := 6$

$munka_j_1 := 5$

$szünet_j := 17$

$munka_j_2 := 5$



1.1.2. 1. ábra

A beágyazás menete. Az *i.* munka a beágyazó *j.* a beágyazott

$$(szünet_j - munka_i_2) / (munka_i_1 + munka_j) = 11 / 34 = 0.32$$

Másik beágyazás:

$munka_i_1 := 7$

$szünet_i := 10$

$munka_i_2 := 6$

$munka_j_1 := 4$

$szünet_j := 17$

$munka_j_2 := 7$

$$(szünet_j - munka_i_2) / (munka_i_1 + munka_j) = 11 / 35 = 0.31$$

Látható, hogy a második esetben, bár hosszabb beágyazásunk (35) van mégis a hozzá mért, keletkezett szünet arányában kisebb, azaz, hogyha például a valós (keletkezett) szünetek egyenlőek, (a példánkban 11) akkor az algoritmus a nagyobb összmunkát válassza, a $35 - t$, hiszen ennél kevesebb a mért arány, ennél van nagyobb kihasználtság és később elérhetően rövidebb lehet a munkasoron az n munkák összhossza.

És most lássunk az [2] forrásból egy újabb heurisztikát, csak hogy lássuk, mennyire függenek a Páros munka ütemezés heurisztikáinak a hatékonysága a bemeneti adatok tulajdonságaitól.

1.1.3 Újabb heurisztikus megoldás egyedi bemenetekre

Az algoritmus munkái egy életszerű tulajdonságon alapszanak: legyenek a munkák részmunkái egy egységnyiek: $a_i = 1$, $b_i = 1$ és a szünetek változó hosszúak: l_i .

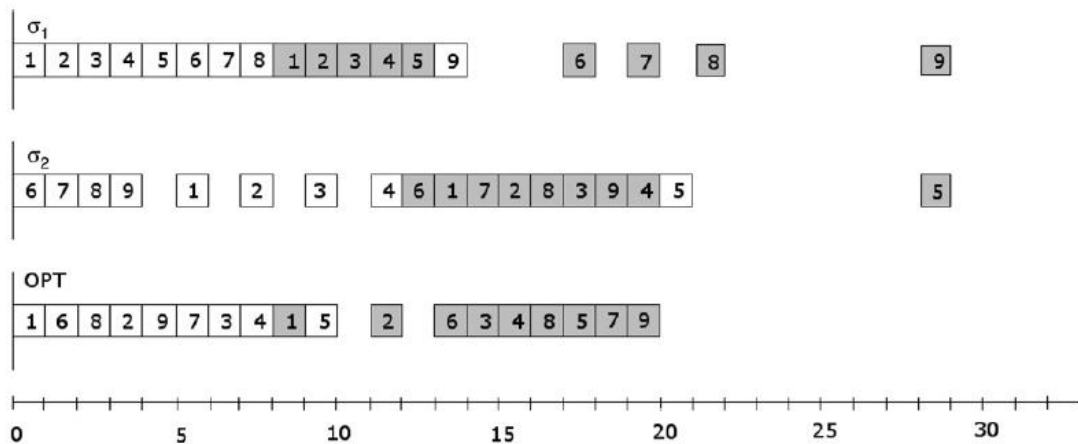
Az algoritmust egyébként DNF- nek, azaz Double – next - fit algoritmusnak is hívják, mert két ütemben két sorozatot készít és legrövidebb idő alatt befejeződöttet választja:

Előkészítés: rendezzük a munkákat szünet szerint nem csökkenő sorrendbe:

$l_1 \leq l_2 \leq \dots \leq l_n$.

1. Alkossuk meg a q_1 elrendezést a következőképpen:
 - 1.1. Helyezzük el az 1. munkát a 0.pozícióba!
 - 1.2. For: $i = 2 \dots n$ indítsuk az i . munkát a legkorábbi helyről, közvetlenül az előzően besorolt munka után.
2. Legyen k az a szám, amely azoknak a munkák számát jelöli, amelyek teljesen befejeződtek az n . munka előtt.
3. Alkossuk meg a q_2 elrendezést a következőképpen (csak ha $k > 0$):
 - 3.1. Indítsuk a $k+1$ munkát a 0. pozícióban.
 - 3.2. For $i = k + 1, \dots, n, 1, \dots, k$: rakjuk az i . munkát a legkorábbi helyre, közvetlenül az előzően besorolt munka után.
4. Az eredmény a q elrendezés a q_1, q_2 közül az, amelyik hamarabb véget ér.

Íme a fentiekről egy szemléltető ábra a [3] –es forrásból:



1.1.3 1. ábra a DNF algoritmus q_1 és a q_2 elrendezése, és optimális megoldása

Következőekben ismertetek néhány közel egzakt és egy egzakt eredményt szolgáltató algoritmust, amelyek kombinatorikus feladatokra nagyon jó alkalmazható korlátozás és szétválasztás módszerét alkalmazzák. Lássuk a módszer lényegét.

1.2 Exponenciális futási idejű algoritmusok. Korlátozás és szétválasztás módszere (branch & bound, B&B)

1.2.1 Branch and Bound eljárás ismertetése

A módszer tárgyalásához tekintsük az alábbi problémát:

(A módszerről adott leírást, bővebben a [1] forrásban olvashatunk.)

$$\min \{ z(x) : x \in L \} \quad *$$

Ahol L azonos dimenziójú, egész koordinátájú, nem negatív, vektorok véges és nem üres halmaza. Adott feltételek mellett a * alatti feladatnak nyilvánvalóan létezik

optimális megoldása, ugyanis véges sok célfüggvény érték minimumát keressük. Legegyszerűbb megoldási módszernek első közelítésben az tűnhetne, hogy L elemeit végigvizsgáljuk, és vesszük egy olyan elemét, amelyen $z(x)$ a legkisebb értéket veszi fel. Ezzel egy optimális megoldáshoz jutunk. Az ilyen típusú eljárást szokásos leszámítási eljárásnak nevezni. A különböző gyakorlati problémáknál L elemszáma igen nagy lehet. Például $|L|=n!$ Így célszerű minél kevesebb lehetséges megoldást megvizsgálni. Ez utóbbi célokat igyekszik realizálni a korlátozás és szétválasztás módszere.

Az eljáráshoz szükséges két függvény, módszer, a további fejezetekben a módszer szót fogom használni, mert ezek a függvények sokszor felfoghatók úgy, mint egy ügyes ötlet, kezdeményezés, a továbbiakban ezért szorítkozok inkább a módszer megnevezéshez. Külön fogom jelezni a két függvénynél, hogy melyikre hivatkozok a szó használatakor. A kombinatorikus megközelítéshez is jobban illik ez a kifejezés.

Ezek egyike az úgynevezett szétválasztási függvény, amely az L halmaz tetszőleges $|L'| > 1$ részhalmazához hozzárendeli L' egy valódi osztályozását. A másik függvény, az úgynevezett korlátozó függvény, amely L tetszőleges L' nem üres részhalmazához hozzárendeli a $z(x)$ ($x \in L'$) függvényértékek egy alsó korlátját. Speciálisan, $L' = \{x\}$ esetén a $z(x)$ függvényértéket. A módszer egy fát épít a fenti szabályokat betartva, amely eljárás során építendő fára több példát is látni fogunk a továbbiakban, és ott megértjük a működését.

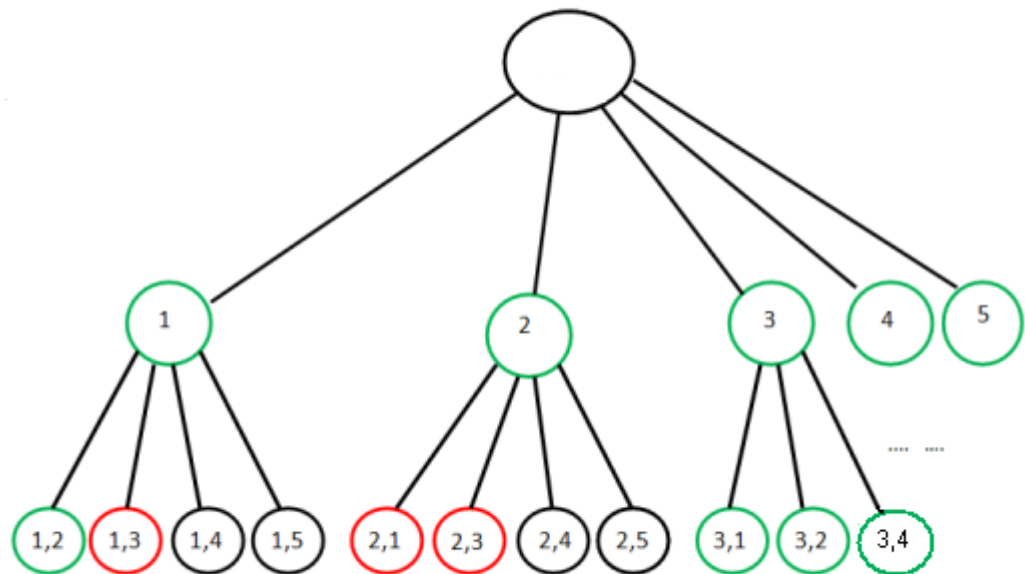
1.2.1. 1. Algoritmus

Ez a megoldási fajta már jóval egzaktabb és két szempontból is újdonság az eddigiekhez képest. Itt nem csupán a munkának az első részfeladatát tudjuk beágyazni a szünetbe - mivel itt már jóval eltérőbb hosszúságú részfeladatokat és szüneteket feltételezünk - hanem egy egész munkát, munkákat, azok részét, ahogy tetszik. Ha veszünk egy sorrendet, ami tudatosan lett választva, később látjuk hogyan, akkor ennek egy lefektetését kell elképzelni. Vagyis oda tesszük a következő munkát ahol a sorrendben legelőször elfér, mindig az elejétől vizsgálva a munkák sorát. Ez egy mohó algoritmus.

Majd ott van a második újdonság, a sorrendje a lefektetésnek. Ennek lényege, hogy az $(1 \dots n)$ számokból álló, összes létrehozható halmazon próbálom ki a működést,

Páros munka ütemezés probléma algoritmusai

kombinatorikus módszerekkel való korlátozással. A halmazokat folyamatosan az 1 eleműtől az n eleműig bővíttem, a következő megoldási fa segítségével, tegyük fel az egyszerűség kedvéért, hogy $n = 5$:



1.2.1. 1. ábra:

A halmazok építése. Az páros munka ütemezés 1. heurisztikus, exponenciális futási idejű algoritmus B&B fája. A piros színű levelek kerülhetnek levágásra, mert a hosszuk több mint az azonos elemeket tartalmazó társai.

Az ábrán jelzett (1,2) és (2,1) lefektetésekről azt feltételezzük, hogy az előbbi (zöld) ad rövidebb összmunkahosszt, és azt a csúcsot építjük tovább a fában, a másikat pedig lezárjuk, természetesen a feltételezéseket pontos számítás helyettesíti, azaz, hogy meddig tart az utolsó munka befejezése a levélben. Ugyanígy az (1,3) és a (3,1) közül azt feltételezzük, hogy a (3,1) lefektetés a rövidebb. Így nem számítva a levágott csúcsokat a fa építésének műveletigénye $O(2^n)$ - melynek igazolását lásd a 6. fejezetben. Persze ehhez az értékhez, az algoritmus teljes futási igényének megállapításához, további szorzókat is hozzá kell tenni, például a fa építésénél a csúcspontok meghatározása is n függvénye.

Íme, az algoritmus:

Páros munka ütemezés probléma algoritmusai

Az páros munka ütemezés 1. exponenciális futási idejű, heurisztikus, korlátozás és szétválasztás algoritmusai:

Előkészítő rész:

Legyen a fa gyökér csúcspontja az üres halmaz.

Iterációs rész:

1. Ha a levelek n elemű halmazok, akkor a gépen az első részmunkaidő kezdetétől az utolsó befejezéséig eltelt idő adja a hosszát a levélnek. Ezek közül a legrövidebb adja az algoritmus megoldását. Ha nem, akkor a 2. lépés következik.

2. Bontsuk a leveleket tovább, mindig ugyan azon a szinten maradva! A következő levél legyen olyan halmaz, ami a szülőjétől +1 elemmel többet tartalmaz és különböző ez az elem a szülőben lévőktől, és szúrjuk a szülő halmaz elemei sorrendjének a végére (1. ábra), a következő módon: az ábra egy olyan algoritmust ír le, ahol a következő munkát - az elejétől vizsgálva a sort - a legelső olyan helyre illeszti be, ahol az elfér.

3. Ha a 2. lépés következtében kialakul a levelek közt azonos elemeket tartalmazó halmaz, akkor az 1. lépésben leírtaknak megfelelően megállapítjuk a hosszakat, és a hosszabbat levágom, egyenlőség esetén mindegy melyiket vágom le. Ezután újra az 1. lépés.

1.2.2. 2. Algoritmus

Az algoritmus hasonló a fentihez, ugyancsak egy branch - and - bound fát épít, csak egy ponton változik a működés. A működés mohó jellegű, úgy, mint az előző heurisztika. Arra hívatott, hogy minél inkább a célra törekedjen, minél hamarabb letenni a munkát, és ezt még a lenti pirosan jelölt szabállyal is jobban kikényszeríthetjük, erőltethetjük.

Íme:

Az páros munka ütemezés 2. exponenciális futási idejű, heurisztikus algoritmusai:

Páros munka ütemezés probléma algoritmusai

Előkészítő rész:

Legyen a fa gyökér csúcspontja az üres halmaz.

Iterációs rész:

1. Ha a levelek n elemű halmazok, akkor a gépen az első részmunkaidő kezdetétől az utolsó befejezéséig eltelt idő adja a hosszát a levélnek. Ezek közül a legrövidebb adja az algoritmus megoldását.
Ha nem, akkor a 2. lépés következik.
2. Bontsuk a leveleket tovább, mindig ugyan azon a szinten maradva!
A következő levél legyen olyan halmaz, ami a szülőjétől +1 elemmel többet tartalmaz és különböző ez az elem a szülőben lévőktől, és szúrjuk a szülő halmaz elemei sorrendjének a végére (1. ábra), a következő módon: az ábra egy olyan algoritmust ír le, ami a következő munkát - az elejétől vizsgálva a sort - a legelső olyan helyre illeszti be, ahol az elfér. **Ezt a munkát csak akkor teszi be erre a helyre, ha azt balra igazítva el tudja helyezni, nevezetesen, ha a munka első részmunkája elhelyezhető közvetlenül egy előző munka részmunkájának befejezése után, kimaradó hely, részszünet nélkül.**
3. Ha a 2. lépés következtében kialakul a levelek közt azonos elemeket tartalmazó halmaz, akkor az 1. pontban leírtaknak megfelelően megállapítjuk a hosszakat, és azt levágom, egyenlőség esetén mindegy melyiket vágom le. Ezután újra az 1. lépés.

2. Egy optimális eredményt szolgáltató algoritmus, hasonló, de más B&B fával, heurisztikus módszereket alkalmazva, amikkel a fa mérete csökkenthető.

Ismertetésre kerül egy optimális megoldást szolgáltató algoritmus, ami az eddigiektől eltérő heurisztikus szétválasztó és korlátozó módszert használ.

Az optimális algoritmusoknál, a jelen esetben, azért nevezhetők heurisztikusoknak a szétválasztó és korlátozó módszerek, mert nem tudni, hogy a szétválasztó módszerek pontosan hány db levelet fognak létrehozni, a korlátozó

módszerek pontosan hány levelet fognak levágni a B&B fa egy szintjén. Ismertetésre kerül először a szétválasztó módszer, majd két heurisztikus korlátozó módszer, végül két egzaktabb, de még mindig heurisztikusnak mondható korlátozó módszer, ez utóbbiakban az előző algoritmusok szerint történik a levágás, pontosabban megjósolható, hogy mennyi levél marad, csak más módon számítja ki a levelekhez rendelt értéket. A levágások a fent említett szétválasztó módszerhez igazodva fognak történni. A heurisztikus korlátozások viszonylag egyszerű feltételek teljesülése esetén használhatók.

Az algoritmus az optimális megoldást adja, csak az eddigiektől kicsit eltérő, sajátos levél kiépítést (szétválasztó módszert) használ, és korlátozó módszert. Az algoritmusról bővebben a [4] forrásban olvashatunk.

2.1 A szétválasztó módszer

Az algoritmus sajátos módon építi ki a leveleket. Az 1. ábrán látható módon, egy halmaz, egy elemmel való bővítését 3 felé bontja, és így elméletileg három levelet készít belőle.

A következő munka beszúrása, a levélben az utolsó munka 1. részmunkája után történik a munkák sorában.

1. eset: amikor a munka szünetében szerepel a teljes új munka, ha elfér.
2. eset: amikor a munka szünetében csak az első részmunkája szerepel az új munkának, ha elfér.
3. eset: amikor az új munka teljes egészében (1. és 2. részmunkája is) a sorban a munka után szerepel.

A 1. ábra jól szemlélteti az algoritmust.

2.2 Egy korlátozó módszer, az azonosság

Páros munka ütemezés probléma algoritmusai

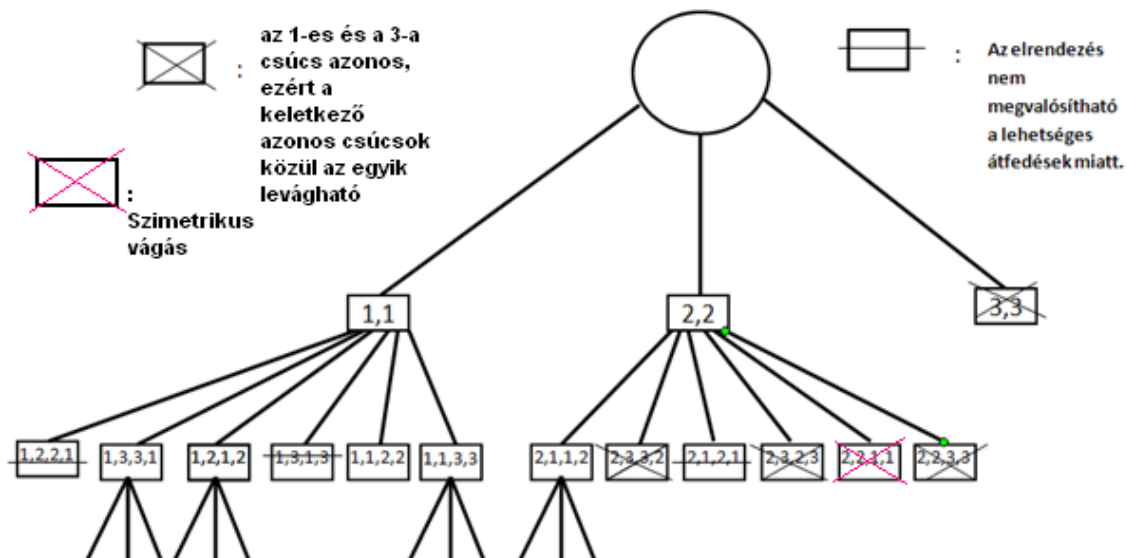
Ha van két olyan munkánk, amely mind a három részükben (1. 2. részmunka és köztük a fix szünet) megegyezik, akkor kialakulhat, úgynevezett azonos munkákat tartalmazó levél. Például, ha az (1' 2' 1'' 2'') és a (1' 3' 1'' 3'') esetet vesszük és a 2. és a 3. munka a fent említett módon teljesen azonos, akkor az egyik levél, bármelyik levágható, hogy miért?

Két azonos munka beszúrása az adott levelekbe, felcserélhető levelekhez juttat, ha ugyan olyan sorrendben lévő elemek közé szúrjuk őket, ahogy a következő példában is látjuk.

Az 1. ábrán jól látható, hogy az 1. és a 3. munka megegyezik, akkor a 2,1,1,2 és a 2,3,3,2 levél közül az egyik levágható pl. a 2,3,3,2. Ugyanolyan kontextusban szerepelnek és ezért, a továbbiakban mindegy, hogy az 1. vagy a 3. munkát használtuk először.

A azonosság számától függően gyorsabb futást érhetünk el algoritmusunkkal.

A módszerről a [7] forrásban olvashatunk.



2 1. ábra Azonos levágás és az ezt lehetővé tevő B&B fa eddigiektől eltérő kiépítése, valamint a symmetry breaking.

2.3 Egy másik korlátozó módszer, a symmetry breaking

Az előző boundon érezhető, hogy nem vezet, előre, számszerűen, pontosan megjósolható levágásokra. Ilyen a szimmetry breaking levágás is. A módszer zseniális, további hasonlóknak nevezhető leveleket lehet levágni vele, a fa építése során, - amit egyszer kiszámoltál, azt még egyszer ne számold ki, fontos informatikai ökölszabálynak eleget tevő módon.

A módszerről a [8] forrásból olvashatunk többet. Vegyük a következő leveleket:

(1,2,3,3,1,2,5,4,4,5) (5,4,4,5,1,2,3,3,1,2,)

,vagy akár hosszabbakat is vehetünk:

(3,6,6,7,8,8,3,7,4,5,5,2,2,4) (4,5,5,2,2,4,3,6,6,7,8,8,3,7)

,vagy csak egyszerűeket véve:

(3,3,4,4) (4,4,3,3)

,de lehetnek összetettebbek is:

(5,6,6,7,8,7,5,8,4,4,3,2,3,2),(4,4,5,6,6,7,8,7,5,8,3,2,3,2),

(3,2,3,2, 4,4,5,6,6,7,8,7,5,8,), ...

Jól látható a különböző árnyalatokkal jelölt egyezések, amik a következőeket jelentik:

Ha van két olyan levél, amely azonos munkákat tartalmaz, akkor megvizsgálható a szimmetrikus levágás lehetősége. A példákban jól látható az azonos árnyalattal jelölt blokkok, amelyek azonos munkákból állnak és azonos elrendezésben, és ezek a blokkok nem tartalmazznak átfedést egymással. A bennük lévő munka mindkét részmunkáját tartalmazza. Nem csak két blokk, de több blokkok esetén is vágható az egyik vagy több

levél. Itt a konzisztencia megőrzése érdekében érdemes egy stratégiát választani, amelyet követve hagyjuk meg a levelet, a többit pedig levágjuk. Tartsuk meg azt, amelyekben a blokkok lexikografikus elrendezést követnek. A fenti példákat követve ezek a következők:

(1,2,3,3,1,2,5,4,4,5), (3,6,6,7,8,8,3,7,4,5,5,2,2,4), (3,3,4,4),

(3,2,3,2, 4,4,5,6,6,7,8,7,5,8,)

2.4 Korlátozás egzaktabb fajtái

Megemlítendő az egzakt megoldásnál további 2 heurisztikus korlátozó módszer. Bővebbet a [6] forrásból olvasható. Az új munkát tudjuk, hogy a sorban az utolsó munka 1. részmunka befejezése után fogjuk beilleszteni.

Nevezzük ezt a munkát T – nek, az 1. részmunkáját $T1$ – nek, a 2. részmunkáját $T2$ – nek.

Tudjuk, hogy már $T1$ előtt, a levélben nem változnak sem a munkák sem a kezdésük és befejezésük időpontja. Tehát ebből a permanent tényből kiindulva felépíthetünk egy alsó határt a szinten, az azonos elemeket tartalmazó levelekhez.

A számítás:

Vegyük a $T1$ befejezési idejét a levélben, majd ehhez adjuk hozzá azokat a részmunka időket, amelyek $T1$ után vannak, azok végrehajtási idejét. Végül ehhez adjuk hozzá még az összes olyan munka részmunkaidejét, amelyek a levélben nincsenek lefektetve még. Ezt a bound – ot trivial boundnak nevezi a forrás.

A következő heurisztika nagyon hasonló az előzőhöz: Nem a $T1$ - től hanem a $T2$ – től fogom számolni a bound – ot. Veszem a befejező idejét a $T2$ – nek és ehhez adom hozzá az összes részmunkát, amely le van, és amelyik még nincs lefektetve a levélben $T2$ után, azok végrehajtási idejét.

3. Az eddigi megoldások előzetes értékelése I.

A pontosabb értékelés előtt lássunk egy elméleti áttekintést, jellemzést az eddigi heurisztikus és optimális megoldásokról, futásainak jellemzéséről. A polinomiális futási idejű algoritmusok, azaz sorban az első kettő leginkább hasonló részmunkaidejű és szünetű munkák esetén hatékonyak, - adnak az optimális megoldástól való eltéréshez képest gyors futási időt.

Most nézzük az exponenciális futási idejű, 2 heurisztikus, és az optimális megoldást szolgáltató algoritmust. A tesztek során kiderült, hogy az esetek többségében a mohó algoritmusok relatíve gyors futás mellett is nagyon sokszor az optimális megoldást adják. Például mikor a munkák nagyon szorosan helyezkednek el, azaz az optimális megoldásban minimális, kevés részszünetek alakulnak ki. Ilyenkor általában a mohó algoritmusok nem tévedhetnek sokat vagy egyáltalán nem, mert nem kell sokat variálniuk a szünetekkel. Szinte minden munka egymás után következik így a mohó választás általában jó lehet. Lássunk erre egy példát az algoritmusból, egy képet és alatta egy szimulációt:

Páros munka ütemezés probléma algoritmusai

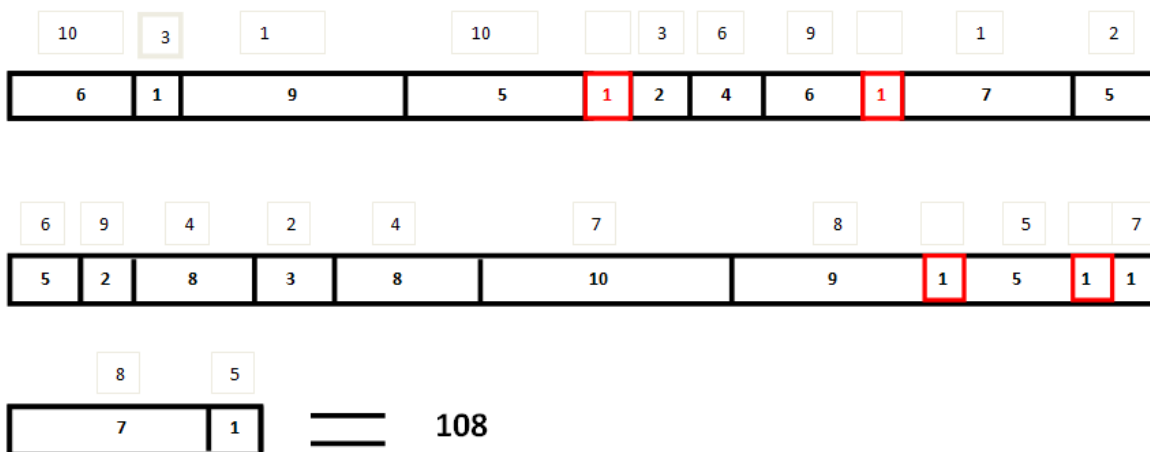


3 1.ábra a páros munka ütemezés heurisztikus, exponenciális futási idejű algoritmus

Páros munka ütemezés probléma algoritmusai

1:9,19,7 2:5,15,3 3:1,15,2 4:8,3,8 5:5,9,1 6:4,19,5 7:10,16,1 8:9,8,7 9:6,18,2 10:6,10,5

10 -> 3 -> 1 -> 6 -> 2 -> 9 -> 4 -> 7 -> 8 -> 5 = 108 egység



3 2. ábra a heurisztikus megoldás is szolgáltathat optimális eredményt, ha az kevés, kicsi részzszüneteket tartalmaz (piros kockák)

A Páros munka ütemezés egyébként az NP – nehéz problémák közé tartozik. Nem feltétlenül lesz a futási idő a bemenet exponenciális függvénye. Ez sok mindentől függ, függ a bemenet tulajdonságaitól, a használt algoritmusban megvalósított technikáktól, jelen esetben a branch – tól és a bound – tól.

Az utóbbi általában, az NP – nehéz problémák esetén erősen heurisztikus, mint például az említett Symmetry breaking, vagy az azonos munkáknál alkalmazott levágás, nem úgy, mint a következő példánál.

Pillantsunk bele egy kicsit egy ugyancsak B&B eljárást alkalmazó problémába az (TSP) Utazó ügynök problémájába. Ott azt kell belátni, hogy a levágások száma, a kiépített csúcsok száma, pontosan, számszerűen kiszámolhatóak, a probléma, a bemenetek kevésbé komplexebb volta miatt. Kombinatorikus előszámításokkal kiszámolható, hogy hány darab új levelet és levágást eredményez az algoritmusunk, csak úgy, mint a két, már bemutatott exponenciális futási idejű, mohó algoritmusaink esetén. Lássuk:

4. Egy iskolaszerű példa az Utazó ügynök problémája (TSP)

Adva van n város illetve az útiköltség bármely két város között, keressük a legolcsóbb utat egy adott városból indulva, amely minden várost pontosan egyszer érint, majd a kiindulási városba ér vissza. Ez egy Hamilton kör.

A különbség az adott n elemet tartalmazó levél létrehozása után az, hogy mögé kell tenni még az 1 – es kiindulási pontot, hiszen így lesz egy Hamilton kör a megoldás.

További kisebb technikai különbség még, hogy két halmaz összehasonlíthatóságához, szükséges még, hogy a levél elemei sorrendjének végén szereplő elemek megegyezzenek (1. ábra). Így nagyobb lesz valamennyivel a fa, kevesebb a vágás. A témáról bővebben a [9] forrásból lehet tájékozódni.

A TSP esetén azt kell látnunk, hogy minden előre adott. Egyetlen értéket takar egy újabb elem, két elem közti távolságot, azaz az út hosszát. Egyszerűbben, kevesebb levél épül ki egy szinten, nem úgy, mint a Páros munka ütemezésnél, hiszen ott az említett, három féle módon helyezkedhet el az új munka és még így sem végleges a levél értéke, a munkák eltologathatása miatt.

A TSP esetén jól működik az elemek egymás mellé pakolásából fakadó, egyszerű összeadásos hossz keresése, mert egy elem egy tulajdonsággal, egyetlen hosszal rendelkezik, azaz a két város közötti távolsággal.

Nézzük most már a TSP B&B faépítő algoritmusát:

Előkészítő rész:

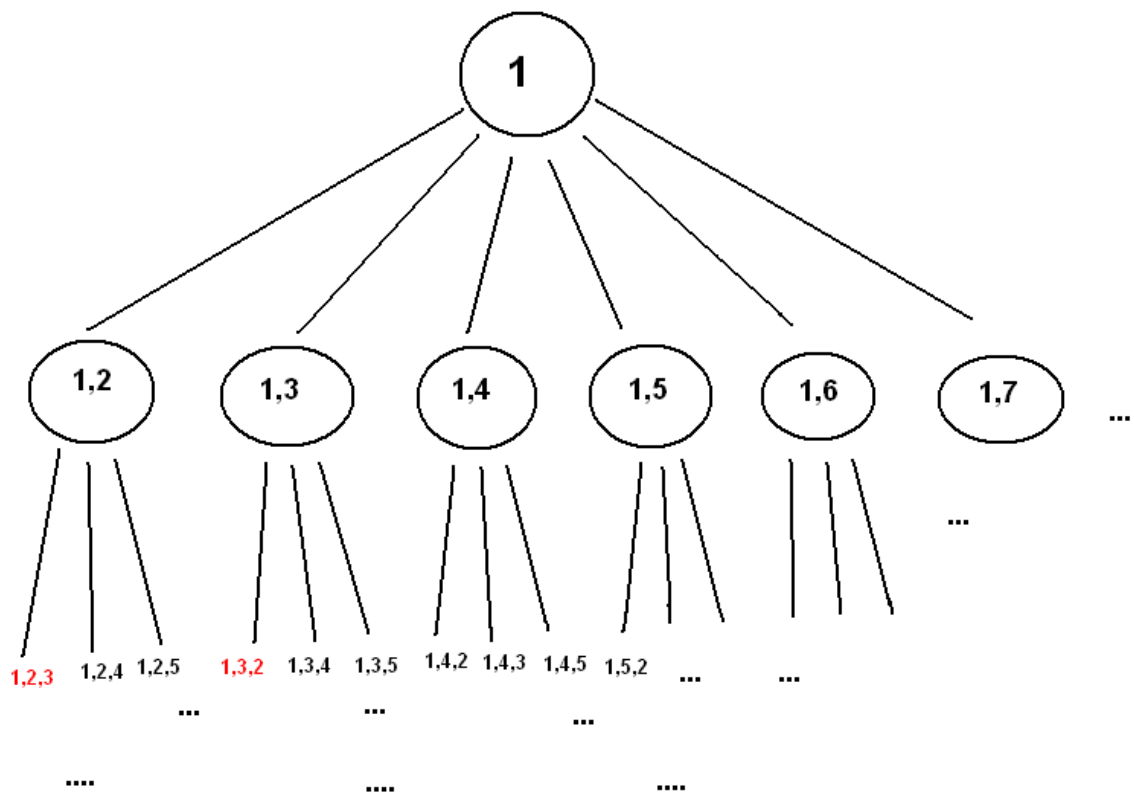
Legyen a fa gyökér csúcspontja a kiindulási csúcspont az egyes (1 elem).

Iterációs rész:

1. ha a levelek n elemű halmazok és minden szülő gyerekeit már meghatároztuk, akkor minden levél végéhez újra szúrjuk az 1 – es elemet így lesz az útból kör és így állapítsuk meg a hosszakat. A legrövidebb fogja adni az optimális megoldást. Ha nem akkor 2. lépés.

Páros munka ütemezés probléma algoritmusai

2. Bontsuk a leveleket tovább! A következő levél legyen olyan halmaz, ami a szülőjétől plusz egy elemmel többet tartalmaz a halmazban még fel nem használt elemek közül (1. ábra), és szűrjük a szülő halmaz elemei sorrendjének a végére.
3. ha a 2. lépés következtében kialakul a levelek közt azonos elemeket tartalmazó halmaz, és az utolsó elem is azonos az elemeinek sorrendjében, akkor meghatározzuk a hosszabbat és levágjuk, egyenlőség esetén mindegy melyiket vágjuk le. Ezután újra az 1. lépés.



4 1. ábra TSP B&B fája

Az (1,2,3) levél és az (1,3,2) levél közül egyik sem kerülhet levágásra, mert nem egyeznek meg az utolsó elemekben, hiába tartalmazzák ugyan azokat az elemeket.

5. Az eddigi megoldások előzetes értékelése II.

A PMÜ lassabban futó algoritmus, az optimális megoldást szolgáltató algoritmus, akkor előnyös leginkább, ha szabad szemmel erősen nem látható, megbújó komplexitás van a bementi munkánk tulajdonságain. Már ilyen példát sem egyszerű találni. Általánosságban, a problémát a kétfajta algoritmus között, mármint az exponenciális mohó és az optimálisat szolgáltató között, egyszerűen magyarázhatjuk: ha nagyok a munkák szünetei, jobban szerepet játszanak a szünetekbe beágyazott munkák elhelyezkedése. Nem működhet az, hogy mindjárt a legelső szabad helyre, balra igazítva teszem le a következő munkát. Íme, egy szemléletes példa:

Legyenek a munkák száma négy, a következők:



5 1.ábra Speciális eset munkái

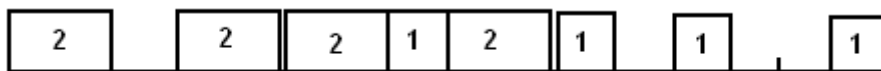
Páros munka ütemezés probléma algoritmusai

Az első ránézésre is jól látható, hogy egymásba ágyazható néhány munka. Vegyük először a heurisztikus algoritmust. Azt tudjuk, hogy a következő munkát a levélben, az elejétől indulva, mohó módon, az első lehetséges helyre teszi. Így a következő rendezés alakul ki:



5 2. ábra A mohó algoritmus működése egy igen speciális esetre, nem adja ki az optimális megoldást.

Most pedig lássuk, mit tesz az optimális algoritmus, amely már több lehetőséggel számol, számba veszi a lefektetés nem csak mohó változatait is. Látni kell a következő 3. ábrán, hogy a második munka lefektetésénél az eggyel jobbra tolt változata miatt jut hely az utolsó munkának, hogy hamarabb elkezdődhessen, íme:



5 3. ábra Az optimális megoldás a speciális esetre

Az első esetben 20 az eredmény a másodikban 16, íme, a 4. ábrán a program futása grafikusán is megjelenítve.

Páros munka ütemezés probléma algoritmusai



5 4. ábra a mohó és az optimális megoldást szolgáltató algoritmus futási eredménye a programban

6. Az 1. exponenciális futási időjű, heurisztikus algoritmus elemzése kombinatorikusan

Az algoritmus futási ideje biztosan $O(2^n)$. Ehhez jön hozzá az n^2 szorzó, amely egyéb, kisegítő mellékszámítások miatt adódnak hozzá, úgymint például a szülő gyerekének, a levelek kialakításából. Így tehát $O(n^2 2^n)$ – val kell számolnunk.

Minden szinten a levelekben szereplő elemeknek, a szülő csúcsa miatt, van egy kezdő, kötött sorrendjük. Ennek következtében minden szülőnek a gyereke eggyel kevesebb darab számú, mivel folyamatosan csökken az n elemű halmaz:

0.szint: gyökerácsúcs: 1 üres halmaz

Páros munka ütemezés probléma algoritmusai

Ezután következnek a különböző szintek:

1. szint $\binom{n}{0}n$ csúcs

2. szint: $\binom{n}{1}(n-1)$ csúcs

.

.

.

$n-1$. szint: $\binom{n}{n-2}2$

n . szint: $\binom{n}{n-1}1$

Összefoglalva az eredmény, a valódi összeg:

$$\sum_{k=1}^n \binom{n}{k} k = n2^{n-1}$$

Felülről becsülve a futási idő viszont

$O(n^2 2^n)$ és nem $O(n2^n)$. Miért n^2 ? Ez azért van, mert a csúcsok kiépítésénél alkalmazott, egymásba ágyazott for ciklusok számba vetése is ezt az eredményt szorgalmazza:

for $i:=1$ to n do

 for $k=1$ to $\text{sizeof}(\text{levél})$ do

 az i tartalmazza az alaphalmaz $(1..n)$ összes elemét

 a k pedig a szülő halmaztömbjének indexe és azt vizsgáljuk, hogy az i benne van-e a halmazban.

Ennek futási ideje:

$$\sum_{k=0}^n n-k = n + n-1 + \dots + n-(n-1) = \frac{n(n+1)}{2}$$

Minden szülő egy gyermeke létrehozásához - átlagosan számítva - ennyi futás kell, és ha számításba vesszük az összes szülőt, amely tudjuk már, hogy 2^n , akkor már biztosabban meghatározhatjuk, és kapjuk az

$$O(n^2 2^n) - t.$$

7. A leimplementált algoritmusok összehasonlítása, összefoglalása

A polinom algoritmusok:

Itt a két algoritmus fej - fej mellett halad, hol az egyik előzi meg sokkal a másikat, hol fordítva, ezért nem kizárt egy olyan tanuló algoritmus, amely megállapít egy - egy eredményt mind a két algoritmuson és a jobb eredményt választja. A legtöbb eltérés akár 20 – 25% is lehet.

Lássuk a két mohót:

Ritkán előfordul, hogy a második algoritmussal egy – két egységnyi javulást kapunk ($n=10$, max (részmunka) = 19 egység) az előző algoritmushoz képest. A jellemző inkább az, érdekes módon, hogy az előző, kötetlenebb ad jobb eredményt, a másik meg sokkal elmarad.

És most a polinom algoritmusokat hasonlítsuk össze a mohó algoritmusokkal:

A futási idő az előbbieknél a másodperc tört része, az utóbbiaknál 1 – 3 mp. De viszont ha ezt a kis időt kivárjuk, a példától függően 0-55 % javulást érhetünk el az eredményben.

Végül az egzakt algoritmus: Tapasztalat szerint már lassabban futó algoritmusról beszélünk, már $n=6$, $n=7$ munkánál is előfordulhat hosszú másodpercekig tartó futás, még akkor is, ha egy boundot, például a trivial boundot használjuk. Az alábbi táblázatban szerepel egy bound nélküli, $O(n!)$ futási idejű leszámlálási algoritmus eredményei is. Szükség volt rá, mert nem volt megbízható a leimplementált trivial bound, holott a táblázatban szerepelt esetekre, mind helyes eredménnyel szolgált. A

Páros munka ütemezés probléma algoritmusai

lassú futás, egyébként, a hosszú szünetek oka lehet az esetek 98% - ban, a [5] forrás ezt úgy nevezi, hogy kis részmunkáknál lassul a program. A két jellemzés ugyanaz.

Most ismertetésre kerülnek azok a további informatikai eszközök, amelyekkel a vizsgálatokat végeztem.

A használt hardware: három magos ATHLON (3.3 GHz, 2GB RAM) gép.

Használt operációs rendszer: Win Xp

Páros munka ütemezés probléma algoritmusai

A táblázat jelölései:

1. T1: Speciális, cseles eloszlású munkák, nagy valószínűséggel csak az optimális megoldást szolgáltatató algoritmusok adnak rá optimális eredményt.
2. T2: közel azonos alkotó elemekből álló munkák.
3. T3: Nagy szünetekből álló munkák.

	n	Heurisztikus, polinom futási idejű 1. algoritmus (szimpla ágyazás)		Heurisztikus, polinom futási idejű 2. algoritmus (arányokat is használ)		Heurisztikus exponenciális futási idejű algoritmus Mohó 1.		Heurisztikus exponenciális futási idejű algoritmus Mohó 2. (a köv. munkát mindenképp balra igazítja)		optimális eredményt adó közel exponenciális futási idejű algoritmus trivial boundal		optimális eredményt adó algoritmus bound nélkül, így $O(n!)$ a futási idő, így biztos, hogy optimális az eredmény	
		eltérés az opt eredménytől	cpu	eltérés az opt eredménytől	cpu	eltérés az opt eredménytől	cpu	eltérés az opt eredménytől	cpu	eltérés az opt eredménytől	cpu	eltérés az opt eredménytől	cpu
T1	4	25%	0,0	25%	0,0	25%	15	25%	15	0%	15	opt	31
T1	5	13%	0,0	13%	0,0	13%	15	13%	15	0%	15	opt	125
T1	6	8%	0,0	8%	0,0	8%	15	8%	16	0%	46	opt	562
T1	8	3,8%	0,0	3,8%	0,0	3,8%	47	3,8%	47	0%	172	opt	49953
T2	4	0%	0,0	0%	0,0	0%	15	0%	16	0%	16	opt	15
T2	5	0,2%	0,0	0,2%	0,0	0%	16	0,2%	15	0%	16	opt	40
T2	6	0,6%	0,0	0,6%	0,0	0%	16	0%	15	0%	15	opt	266
T2	8	2,8%	0,0	2,8%	0,0	0%	62	0%	62	0%	62	opt	10312
T3	4	58%	0,0	112%	0,0	0%	15	0%	16	0%	594	opt	2422
T3	5	57%	0,0	59%	0,0	11%	15	11%	16	0%	94	opt	182687
T3	6	23%	0,0	23%	0,0	5,4%	15	5,4%	16	0%	1735	opt	142235
T3	8	r:131	0,0	r:132	0,0	r:107	47	r:103	50	--	--	--	--

táblázat 1. a futási eredmények összefoglalója

Irodalom jegyzék:

- [1] Imreh Balázs – Imreh Csanád : Kombinatorikus optimalizálás
135-142 oldal

- [2] József Békési, Gábor Galambos, Marcus Oswald, Gerhard Reinelt
Improved analysis of an algorithm for the coupled task problem with UET jobs;
1-2 oldal.

- [3] József Békési, Gábor Galambos, Marcus Oswald, Gerhard Reinelt
Improved analysis of an algorithm for the coupled task problem with UET jobs;
2. oldal.

- [4] József Békési, Gábor Galambos, Michael N. Jung, Marcus Oswald, Gerhard Reinelt : A Branch-and-Bound Algorithm for the Coupled Task Problem
1 - 5

- [5] József Békési, Gábor Galambos, Michael N. Jung, Marcus Oswald, Gerhard Reinelt : A Branch-and-Bound Algorithm for the Coupled Task Problem
13. oldal

- [6] József Békési, Gábor Galambos, Michael N. Jung, Marcus Oswald, Gerhard Reinelt : A Branch-and-Bound Algorithm for the Coupled Task Problem
3. oldal.

- [7,8] József Békési, Gábor Galambos, Michael N. Jung, Marcus Oswald, Gerhard Reinelt : A Branch-and-Bound Algorithm for the Coupled Task Problem
5. oldal.

- [9] Papadimitriou C.H. Számítási Bonyolultság.
19. oldal

Nyilatkozat

Alulírott szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Tanszékcsoport Tanszékén készítettem, diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Informatikai Tanszékcsoport könyvtárában, a helyben olvasható könyvek között helyezik el.

2013-05-13

Aláírás

Köszönetnyilvánítás

Tisztelettel megköszönöm témavezetőmnek, Dr. Békési József Tanár úrnak a szakdolgozat elkészítésében nyújtott áldozatos, megértő és minőségi segítségét, továbbá köszönetet nyilvánítok külső témavezetőm Dr. Pluhár András felé, akit tanáromként is megismerhettem. Továbbá megköszönöm Dr. Dombi József ugyancsak egykori tanáromnak a közreműködést, hogy megvalósulhatott szakdolgozatom, még egyszer nagyon köszönöm mindháromójuknak.

Dinnyés Balázs.